

Resumen

Este informe documenta el tercer avance del proyecto: la conexión de una placa ESP32 real con el backend de monitoreo mediante WiFi, NTP, HiveMQ Cloud y MQTT sobre TLS. El objetivo de esta etapa fue reemplazar la incertidumbre de una integración únicamente simulada por una línea base física reproducible, sin confundir esa línea base con una validación completa de sensores, tolerancia a fallos o un producto automotriz certificado.

La placa fue identificada por su MAC de eFuse d0:ef:76:45:f6:40. A partir de esa identidad se fijaron vehicle-001 y esp32-d0ef7645f640, se registraron ambos valores en la API productiva y se creó una credencial exclusiva de HiveMQ con una ACL limitada al árbol MQTT del dispositivo. La contraseña no se incluye en este documento ni en el repositorio; permanece en el archivo local ignorado include/secrets.h.

Durante la puesta en marcha se encontraron y corrigieron tres problemas de integración. El primero era un límite de fecha que desbordaba el tipo time_t de 32 bits del ESP32 y rechazaba una hora NTP válida. El segundo era la referencia a un bundle de certificados de Arduino sin inicializar, que impedía validar la cadena TLS de HiveMQ. El tercero era un desajuste entre la contraseña local y la credencial creada en el broker. Después de corregirlos, la sesión serial mostró time_valid=true, conexión TLS a HiveMQ, mqtt_connected=true y acuses PUBACK de telemetría QoS 1.

La API observó el dispositivo en estado online, con firmware 0.2.0 y simulated=false, y devolvió veinte muestras físicas recientes. Esto valida el transporte y la ingesta básica ESP32--HiveMQ--backend--PostgreSQL. No valida todavía los sensores desconectados de la sesión, la microSD, el replay después de un corte, el mensaje LWT offline, la semántica de comandos ni una prueba prolongada de estabilidad. Esas brechas se mantienen explícitas para que el avance sea auditable.

1 Propósito, alcance y método

2 Motivación del avance

El informe anterior describe el despliegue web y las razones para utilizar AWS como plataforma de infraestructura. Ese documento responde dónde se ejecuta la aplicación. El presente informe responde una pregunta distinta: si una identidad física puede atravesar la cadena completa de adquisición, transporte seguro, persistencia y consulta sin recurrir al simulador.

La línea base E2E se diseñó como una prueba de transporte, no como una campaña de caracterización de sensores. La placa se dejó sola en una protoboard, sin conectar los periféricos del prototipo. Esa decisión aisló la comunicación y permitió comprobar que el firmware conserva un dato inválido como inválido, en lugar de convertir la ausencia de un sensor en un cero aparentemente válido.

3 Criterio de evidencia

Se utilizan cuatro etiquetas para evitar mezclar niveles de confianza:

Implementado. Existe código o configuración para la capacidad.

Validado automáticamente. Una prueba de compilación, contrato o suite cubre el comportamiento sin hardware.

Validado físicamente. El ESP32 real produjo la observación durante la sesión fechada.

Validado en producción. La infraestructura productiva devolvió una respuesta o un registro observable asociado a la placa real.

Una captura de la interfaz, una compilación o un resultado del simulador no se interpreta como evidencia física. Del mismo modo, el campo simulated=false significa que el mensaje no fue marcado como simulado; no constituye por sí solo una certificación de procedencia productiva.

4 Arquitectura integrada

5 Flujo de extremo a extremo

La arquitectura separa adquisición, transporte, ingesta y presentación. El ESP32 no conoce PostgreSQL ni expone credenciales MQTT al navegador. HiveMQ Cloud funciona como transporte autenticado; FastAPI es la frontera de confianza que valida tópico, identidad y esquema antes de persistir.

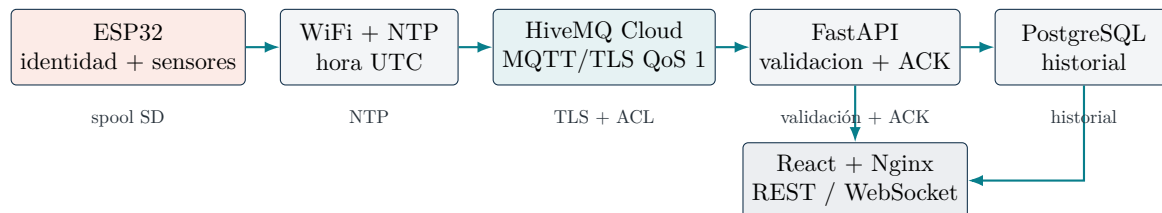


Figura 1: Flujo lógico de la integración validada. Las flechas representan interfaces del sistema; no implican que cada periférico del ESP32 haya sido conectado durante esta sesión.

6 Contratos y tópicos

La versión integrada usa telemetría v3. Cada muestra conserva `vehicle_id`, `device_id`, `boot_id`, `sequence`, `sample_id`, `time_valid`, `replayed` y `simulated`, además de las banderas de validez de sensores. El backend agrega `received_at`, de modo que una hora de medición ausente no se reemplaza por una fecha inventada.

Los tópicos canónicos de la identidad real son:

```

vehiclesense/v1/vehicles/vehicle-001/devices/esp32-d0ef7645f640/telemetry
vehiclesense/v1/vehicles/vehicle-001/devices/esp32-d0ef7645f640/status
vehiclesense/v1/vehicles/vehicle-001/devices/esp32-d0ef7645f640/events
vehiclesense/v1/vehicles/vehicle-001/devices/esp32-d0ef7645f640/acoustic
vehiclesense/v1/vehicles/vehicle-001/devices/esp32-d0ef7645f640/commands
vehiclesense/v1/vehicles/vehicle-001/devices/esp32-d0ef7645f640/command-acks
  
```

El dispositivo publica telemetría, estado, eventos, acústica y acuses; solo se suscribe a `commands`. El backend se suscribe a las ramas de entrada y publica comandos. El estado online se publica retenido y el LWT offline está implementado para una desconexión inesperada, pero la transición LWT todavía no se indujo en la prueba física de este informe.

7 Capas de validación

Antes de la sesión física se mantuvieron las pruebas unitarias de payload, contratos MQTT, colas offline, backend, frontend y simulador. En conjunto, la matriz automatizada registrada en el estado del proyecto reportó 24/24 builds ESP32, 34/34 casos Unity nativos, fixtures contractuales válidos e inválidos, 15/15 pruebas backend y 6/6 pruebas frontend. Estas cifras son una base de software; la sesión descrita en los capítulos siguientes añade la dimensión física.

8 Lectura de identidad

La identidad no se inventó a partir de la etiqueta de la placa. Se leyó el registro eFuse del chip por el puerto serial USB-UART. El resultado identificó un ESP32-D0WD-V3, revisión 3.1, con MAC de fábrica:

```
d0:ef:76:45:f6:40
```

La normalización elimina dos puntos y utiliza minúsculas para formar `esp32-d0ef7645f640`. Esta regla es determinista y permite repetir el registro para otra placa sin reutilizar la identidad del dispositivo anterior. El vehículo se mantuvo separado de la identidad física y se asignó como `vehicle-001`.

Ficha de identidad y transporte.

Chip: ESP32-D0WD-V3, revisión 3.1.

MAC eFuse: `d0:ef:76:45:f6:40`.

`vehicle_id`: `vehicle-001`.

`device_id`: `esp32-d0ef7645f640`.

Firmware: 0.2.0, perfil `vehiclesense_wifi`.

Broker: HiveMQ Cloud, MQTT/TLS TCP, puerto 8883.

Credencial: `esp32-d0ef7645f640`, exclusiva para el dispositivo.

ACL: raíz MQTT del vehículo y dispositivo; sin permiso global `#`.

9 Registro en el backend

Con el auto-registro deshabilitado, se crearon primero el vehículo y el dispositivo mediante la API. Esta secuencia evita que un cliente que conozca un nombre plausible pueda crear identidades espontáneas. El resultado observable en PostgreSQL fue un vehículo y un dispositivo físico asociado. La API devolvió posteriormente el mismo par de IDs, el estado online, firmware 0.2.0, una hora `last_seen_at` y la marca `simulated=false`.

10 Credencial y ACL en HiveMQ

Se creó una credencial propia para el ESP32, separada de la credencial del backend. La permission se nombró `esp32-d0ef7645f640-scope` y limita el acceso a:

```
vehiclesense/v1/vehicles/vehicle-001/devices/esp32-d0ef7645f640/#
```

Esta raíz es una concesión acotada para el prototipo: permite las ramas de telemetría, estado, eventos, acústica, comandos y acuses de una sola identidad, pero no autoriza otros vehículos o dispositivos. La contraseña se probó en el cliente local y no se imprimió en el monitor ni se guardó en Git. El backend conserva otra credencial con sus permisos de suscripción y publicación.

11 Configuración separada de secretos

El archivo versionado `include/secrets_example.h` solo contiene nombres de variables y valores de ejemplo. El archivo activo `include/secrets.h` está ignorado por Git y contiene la red WiFi del hotspot, la credencial HiveMQ y las identidades reales. Esta separación permite compilar el proyecto sin exponer contraseñas en el informe, capturas o commits.

12 Puesta en marcha del firmware

13 Secuencia de arranque

El perfil `vehiclesense_wifi` ejecuta una secuencia deliberadamente conservadora:

1. inicializa la identidad y el registro serial;
2. asocia la interfaz WiFi al hotspot configurado;
3. solicita NTP y espera una hora UTC dentro del intervalo permitido;
4. inicializa ESP-MQTT sobre TLS con la CA compilada;
5. se suscribe a comandos, publica el estado retenido online y comienza la telemetría QoS 1;
6. mantiene la adquisición local aunque un periférico esté ausente.

La lógica no considera suficiente que WiFi esté conectado: el handshake TLS se realiza después de disponer de una hora válida. Esto evita que el certificado del broker sea rechazado por una fecha de 1970 o por una hora todavía no sincronizada.

14 Corrección del tiempo NTP

La primera ejecución solicitaba NTP correctamente, pero la función de validación comparaba el resultado con el límite 4102444800 usando un tipo que en el entorno ESP32 coincidía con `time_t` de 32 bits. El literal se convertía de forma incorrecta y una fecha de 2026 se trataba como inválida. El cambio adoptado fue mantener los límites como `int64_t` y convertir el valor de `time(nullptr)` antes de compararlo:

```
constexpr int64_t kMinimumValidEpoch = 1704067200LL;
constexpr int64_t kMaximumValidEpoch = 4102444800LL;
const int64_t now = static_cast<int64_t>(time(nullptr));
return now >= kMinimumValidEpoch && now < kMaximumValidEpoch;
```

Después de la corrección, el monitor confirmó `time_valid=true`. El backend recibió `measured_at` y conservó por separado `received_at`, como define el contrato.

15 Corrección de la cadena TLS

El segundo bloqueo apareció durante el handshake. El firmware referenciaba el bundle de certificados de Arduino, pero ese bundle no estaba inicializado en este perfil. El serial registró un fallo TLS con `error=32794` y `stack=30336`. Desactivar la verificación no era una opción aceptable: habría ocultado el problema y debilitado el transporte.

Se reemplazó la referencia por el ancla pública ISRG Root X1 en el archivo `TrustedRootCa.h` de comunicación y se mantuvo activa la comprobación del nombre del host. La biblioteca ESP-MQTT siguió usando TLS TCP y el hostname del cluster. La raíz es material público de confianza, no una llave privada; aun así, debe revisarse si HiveMQ cambia su cadena de certificados.

16 Corrección de credenciales

Con el certificado resuelto, el broker todavía podía rechazar una sesión si la contraseña local no correspondía a la credencial recién creada. Se comparó el nombre de usuario configurado con el registro de HiveMQ, se actualizó el secreto local y se observó un `CONNACK MQTT` de código cero en la prueba de diagnóstico. El valor de la contraseña no se reprodujo en ningún log ni en este informe.

17 Carga y monitor serial

La imagen final compiló y se cargó por el adaptador USB-UART identificado como `/dev/cu.usbserial-0001`. El perfil utilizó 24.1 % de RAM y 58.1 % de flash reportados por PlatformIO. El arranque final correspondió al `boot_id=14`. La siguiente transcripción es un resumen de líneas observadas; se omiten credenciales y datos de red no necesarios:

```
boot_id=14
wifi_connected=true
time_valid=true
mqtt_connected=true
HiveMQ connected with TLS; command ACL requested
Telemetry PUBACK token=20
```

La asociación WiFi tardó más que el primer ciclo de diagnóstico y presentó un estado transitorio `wifi_connected=false` alrededor de los diez segundos. No se interpretó ese estado intermedio como fallo definitivo porque el mismo arranque alcanzó NTP, TLS y MQTT poco después.

18 Muestras y respuesta del backend

La consulta productiva de dispositivos devolvió un único registro físico. La respuesta se conserva aquí como una ficha para que pueda compararse con una ejecución posterior:

`device_id:` esp32-d0ef7645f640.

`vehicle_id:` vehicle-001.

`status:` online.

`firmware_version:` 0.2.0.

`simulated:` false.

`last_seen_at:` 2026-09-01T03:55:29.865949+00:00.

La consulta de telemetría con ventana de una hora y límite de veinte devolvió veinte muestras recientes. La última identificada en esa consulta fue `esp32-d0ef7645f640:13:275`, medida a las 03:55:26 UTC y recibida por el backend a las 03:55:26.747851 UTC. El identificador conserva el boot y la secuencia, lo que permite deduplicar una reentrega QoS 1.

19 Cronología resumida

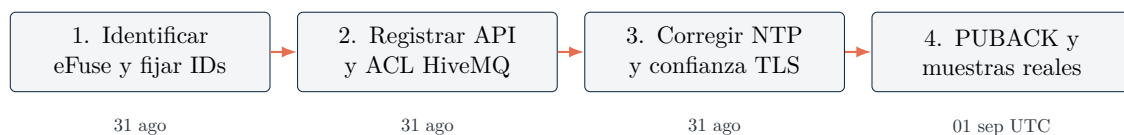


Figura 2: Secuencia de resolución de bloqueos y aceptación del transporte.

20 Conclusiones

Este tercer avance transforma la integración E2E de una hipótesis de software en una observación física acotada. El ESP32 real fue identificado, registrado y autenticado con una ACL individual. La corrección del tiempo NTP y de la cadena de confianza TLS eliminó los dos bloqueos que impedían llegar al broker; la corrección de la contraseña permitió completar el CONNACK. La secuencia final mostró WiFi, hora válida, TLS, MQTT y PUBACK, y la API productiva devolvió el dispositivo en línea y veinte muestras asociadas a los IDs correctos.

El resultado es suficiente para continuar con la línea base física de sensores, fallos, replay y comandos. No es suficiente para afirmar que la PCB está lista, que el sistema es tolerante a fallos, que los datos acústicos están calibrados o que el proyecto constituye un sistema de seguridad vehicular. Mantener esas fronteras explícitas permite que los siguientes informes se apoyen en evidencia fechada y no en capturas o suposiciones.